Machine Learning-Based Fatigue Classification for Tennis EMG

Author: EMG Analysis Team
Date: 2026-03-07
Status: Research Documentation & Implementation Guide
Related: `MOTION_ARTIFACTS.md`, `HIGH_LEVEL_DESIGN.md`, `emg_util.py`

Table of Contents

1. Executive Summary
2. Theoretical Foundation
3. Feature Engineering
4. Labeling Strategy
5. Model Architecture
6. Cross-Validation Strategy
7. Evaluation Metrics
8. Implementation Pipeline
9. Expected Results & Validation
10. Future Enhancements
11. Paper-Ready Methods Section
12. References
13. Code Examples

Executive Summary

Why Machine Learning?

Single-feature fatigue markers (e.g., MDF alone) fail in tennis because:

1. **Non-stationarity:** Tennis swings violate MDF's quasi-stationarity assumption (CV=46.9% vs. required <15%)
2. **Motion artifacts:** While minimal in our data (r=0.006), single features are vulnerable
3. **Inter-subject variability:** EMG amplitude varies 10-100× across individuals

Machine learning reframes fatigue detection as multivariate pattern recognition:

- No single feature needs to be reliable at all times
- Classifier learns feature combinations robust to artifacts
- EMG and IMU features complement each other

Key Advantages

Aspect	MDF-Only	ML Approach
Motion artifact	Breaks	Learned around
Non-stationarity	Invalidates	Implicitly handled
Inter-subject variability	Poor	Normalized + learned
Fatigue manifestation	Single axis	Multi-dimensional

Aspect	MDF-Only	ML Approach

Expected Outcomes

1. **Improved robustness:** 30-50% reduction in false positives vs. MDF-only
2. **Better generalization:** Cross-session accuracy >75% (vs. ~60% for MDF thresholding)
3. **Feature insights:** Identify which features matter most (likely RMS + ENV + one frequency feature)
4. **Publication-ready:** Complete methodology for sports EMG fatigue detection

---

Theoretical Foundation

1. Why Single-Feature Fatigue Markers Fail in Tennis

1.1 Non-Stationarity Violations

MDF requires quasi-stationary signals (Merletti & Parker, 2004):

> "The signal should have relatively constant statistical properties (mean, variance) within the analysis window."

Tennis swings are highly non-stationary:

- Acceleration phase: Low EMG, few motor units
- Impact phase: High EMG spike, many motor units
- Follow-through: Decreasing EMG

Quantitative evidence (from our log0305 analysis):

```
Within-swing RMS CV: 46.9%
Quasi-stationarity threshold: <15% (Merletti & Parker, 2004)
Violation factor: 3.1×
```

Result: MDF computed on non-stationary windows is mathematically unstable, even if motion artifact is absent.

1.2 Motion Artifact Susceptibility

While our motion artifact analysis showed minimal contamination (r=0.006), **single-feature approaches are inherently vulnerable:**

- **MDF drops 10-30%** when motion artifact is present (Clancy et al., 2002)
- **No way to detect** when artifact occurs in single-feature approach
- **No redundancy** if primary feature fails

ML provides robustness:

- Multiple features provide redundancy
- Classifier learns to weight features based on reliability
- IMU features can detect artifact-prone periods

1.3 Inter-Subject Variability

EMG amplitude varies dramatically across individuals:

- Electrode placement variations
- Skin impedance differences

- Muscle mass and fiber type composition
- Subcutaneous fat thickness

Literature evidence:

- Burden (2010): EMG amplitude varies 10-100× across subjects
- Hermens et al. (2000): Normalization essential for cross-subject comparison

Single-feature approach:

- Requires manual threshold tuning per subject
- Fails when subject characteristics change

ML approach:

- Within-subject normalization handles amplitude differences
- Learns subject-invariant patterns
- Generalizes across individuals

2. ML as Multivariate Pattern Recognition

2.1 Feature Complementarity

Different features capture different aspects of fatigue:

Feature Type	What It Captures	Robustness
Time-domain (RMS, MAV)	Amplitude changes	High (robust to spectral shifts)
Frequency-domain (MDF, MNF)	Spectral compression	Moderate (requires stationarity)
Envelope (ENV*mean, ENV*peak)	Activation level	High (simple, direct)
IMU (peak_acc, gyro)	Movement intensity	High (independent of EMG)

Key insight: When one feature fails (e.g., MDF in non-stationary window), others compensate (e.g., RMS, ENV_mean).

2.2 Ensemble Robustness

Random Forest averages predictions from 100-300 decision trees:

- Each tree sees different feature subsets
- Voting reduces variance
- Outlier features have minimal impact

Mathematical basis:

```
Variance reduction: σ²_ensemble = σ²_tree / N_trees
For N=100: ~10× variance reduction
```

Practical impact:

- Per-swing MDF CV = 38.7%
- Expected RF prediction CV ≈ 12% (√10 reduction)

2.3 Literature Support

Phinyomark et al. (2012): "Feature fusion and selection for EMG signal classification"

- Tested 37 EMG features across 6 classifiers
- **Key finding:** Feature combinations outperform single features by 15-40%
- **Best features:** Time-domain (RMS, WL) + frequency-domain (MDF) + AR coefficients

Subasi et al. (2020): "Classification of EMG signals using wavelet neural network"

- Wavelet features + neural network for fatigue detection
- **Accuracy:** 92% (vs. 68% for MDF-only)
- **Key insight:** Multi-scale features capture fatigue better than single-scale

Bonato et al. (2001): "Time-frequency parameters for assessing muscle fatigue"

- Validated activation gating for dynamic contractions
- **Finding:** Gating improves MDF reliability by 25-35%
- **Our implementation:** ENV gating already integrated ✅

Feature Engineering

1. EMG Time-Domain Features

Rationale: Robust to non-stationarity, capture amplitude changes directly.

1.1 Root Mean Square (RMS)

Definition:

```
RMS = sqrt(1/N * Σ(x[i]²))
```

Physiological interpretation:

- Proportional to muscle force (Merletti & Parker, 2004)
- Increases with motor unit recruitment
- Decreases with fatigue (reduced firing rates)

Advantages:

- Simple, fast computation
- Robust to spectral shifts
- Low sensitivity to non-stationarity (CV typically 20-30%)

Implementation:

```
rms = np.sqrt(np.mean(emg_proc**2))
```

1.2 Mean Absolute Value (MAV)

Definition:

```
MAV = 1/N * Σ|x[i]|
```

Physiological interpretation:

- Similar to RMS but less sensitive to outliers

- Correlates with muscle activation level

Advantages:

- More robust to spikes than RMS
- Computationally simpler (no squaring)

Implementation:

```
mav = np.mean(np.abs(emg_proc))
```

1.3 Variance (VAR)

Definition:

```
VAR = 1/N * Σ(x[i] - mean(x))²
```

Physiological interpretation:

- Measures signal power around mean
- Increases with motor unit synchronization

Advantages:

- Captures signal variability
- Complements RMS

Implementation:

```
var = np.var(emg_proc)
```

1.4 Waveform Length (WL)

Definition:

```
WL = Σ|x[i+1] - x[i]|
```

Physiological interpretation:

- Measures signal complexity
- Increases with motor unit firing rate
- Sensitive to frequency content

Advantages:

- Captures both amplitude and frequency information
- Robust to baseline drift

Implementation:

```
wl = np.sum(np.abs(np.diff(emg_proc)))
```

1.5 Integrated EMG (IEMG)

Definition:

```
IEMG = Σ|x[i]| * dt
```

Physiological interpretation:

- Cumulative muscle activation
- Proportional to total work done

Advantages:

- Simple, intuitive
- Already implemented ✅

Implementation:

```
iemg = np.sum(env) * dt
```

2. EMG Frequency-Domain Features

Rationale: Capture spectral compression due to fatigue, but require careful handling of non-stationarity.

2.1 Median Frequency (MDF)

Definition:

```
∫[0 to MDF] PSD(f) df = ∫[MDF to ∞] PSD(f) df
```

Physiological interpretation:

- Shifts from ~150 Hz (fresh) to ~80 Hz (fatigued)
- Reflects motor unit conduction velocity slowing

Advantages:

- Well-established fatigue marker
- Validated in literature

Limitations:

- Requires quasi-stationarity (CV < 15%)
- Our data: CV = 38.7% (violates assumption)

ML integration:

- Include as feature, but don't rely on it alone
- Classifier learns when MDF is reliable

Implementation:

```
_, mdf = mnf_mdf(emg_proc, fs)
```

2.2 Mean Frequency (MNF)

Definition:

```
MNF = Σ(f * PSD(f)) / Σ(PSD(f))
```

Physiological interpretation:

- Similar to MDF, but weighted by power
- More sensitive to high-frequency components

Advantages:

- Complements MDF
- Captures different aspect of spectral shift

Implementation:

```
mnf, _ = mnf_mdf(emg_proc, fs)
```

2.3 Spectral Slope

Definition:

```
slope = d(log(PSD)) / d(log(f))
```

Physiological interpretation:

- Negative slope indicates power concentration at low frequencies
- Becomes more negative with fatigue

Advantages:

- Less sensitive to absolute power
- Captures spectral shape

Implementation:

```
freqs, psd = signal.welch(emg_proc, fs, nperseg=256)
log_freqs = np.log10(freqs[1:])
log_psd = np.log10(psd[1:])
slope, _ = np.polyfit(log_freqs, log_psd, 1)
```

2.4 Total Power (20-450 Hz)

Definition:

```
P_total = ∫[20 to 450] PSD(f) df
```

Physiological interpretation:

- Total signal energy in EMG band

- Decreases with fatigue (reduced motor unit activity)

Advantages:

- Simple, robust
- Less sensitive to spectral shape

Implementation:

```
freqs, psd = signal.welch(emg_proc, fs, nperseg=256)
idx = (freqs >= 20) & (freqs <= 450)
total_power = np.trapz(psd[idx], freqs[idx])
```

3. EMG Envelope Features

Rationale: Capture activation level, integrate with ENV gating.

3.1 ENV Mean

Definition:

```
ENV_mean = mean(envelope(EMG))
```

Physiological interpretation:

- Average activation level during swing
- Correlates with effort

Advantages:

- Simple, robust
- Already implemented ✅

Implementation:

```
env_mean = np.mean(env)
```

3.2 ENV Peak

Definition:

```
ENV_peak = max(envelope(EMG))
```

Physiological interpretation:

- Maximum activation during swing
- Indicates peak effort

Advantages:

- Captures intensity
- Already implemented ✅

Implementation:

```
env_peak = np.max(env)
```

3.3 ENV Area

Definition:

```
ENV_area = ∫envelope(EMG) dt
```

Physiological interpretation:

- Total activation over swing
- Similar to IEMG

Advantages:

- Cumulative measure
- Robust to transient spikes

Implementation:

```
env_area = np.trapz(env, dx=1/fs)
```

4. IMU Kinematic Features

Rationale: Disambiguate effort vs. fatigue, detect artifact-prone periods.

4.1 Peak Acceleration

Definition:

```
peak_acc = max(sqrt(ax² + ay² + az²))
```

Physiological interpretation:

- Maximum acceleration during swing
- Indicates stroke intensity

Advantages:

- Independent of EMG
- Helps distinguish high-effort vs. fatigue
- Already implemented ✅

Implementation:

```
acc_mag = np.sqrt(ax**2 + ay**2 + az**2) / 8192.0  # Convert to g
peak_acc = np.max(acc_mag)
```

4.2 Mean Acceleration

Definition:

```
mean_acc = mean(sqrt(ax² + ay² + az²))
```

Physiological interpretation:

- Average acceleration during swing
- Indicates overall movement intensity

Advantages:

- Less sensitive to transient spikes
- Complements peak_acc

Implementation:

```
mean_acc = np.mean(acc_mag)
```

4.3 Peak Gyroscope

Definition:

```
peak_gyro = max(sqrt(gx² + gy² + gz²))
```

Physiological interpretation:

- Maximum angular velocity
- Indicates swing speed

Advantages:

- Independent measure of intensity
- Already implemented ✅

Implementation:

```
gyro_mag = np.sqrt(gx**2 + gy**2 + gz**2) / 16.4  # Convert to dps
peak_gyro = np.max(gyro_mag)
```

4.4 Angular Jerk

Definition:

```
angular_jerk = d(gyro_mag) / dt
```

Physiological interpretation:

- Rate of change of angular velocity
- Indicates smoothness of swing

Advantages:

- Captures swing quality
- May degrade with fatigue

Implementation:

```
angular_jerk = np.max(np.abs(np.diff(gyro_mag) * fs))
```

4.5 Impact Impulse

Definition:

```
impact_impulse = ∫[t_peak-50ms to t_peak+50ms] acc_mag dt
```

Physiological interpretation:

- Energy transferred during impact
- Proxy for stroke power

Advantages:

- Captures impact characteristics
- May decrease with fatigue

Implementation:

```
i_peak = np.argmax(acc_mag)
win_size = int(0.05 * imu_fs)  # 50ms
i0 = max(0, i_peak - win_size)
i1 = min(len(acc_mag), i_peak + win_size)
impact_impulse = np.trapz(acc_mag[i0:i1], dx=1/imu_fs)
```

5. Feature Normalization

Rationale: Essential for cross-subject learning and generalization.

5.1 Within-Subject Normalization

Method: Median/IQR normalization (robust to outliers)

Formula:

```
x_norm = (x - median_subject) / IQR_subject
```

Why median/IQR instead of mean/std:

- Robust to outliers (e.g., artifact-contaminated swings)
- Better for non-Gaussian distributions
- Recommended for EMG (Burden, 2010)

Implementation:

```
def normalize_features_within_subject(features, subject_id):
    """Normalize features using within-subject median/IQR."""
    subject_data = features[features['subject_id'] == subject_id]

    for col in feature_columns:
        median = subject_data[col].median()
        q75 = subject_data[col].quantile(0.75)
        q25 = subject_data[col].quantile(0.25)
        iqr = q75 - q25

        if iqr > 0:
            features.loc[features['subject_id'] == subject_id, col] = \
                (features.loc[features['subject_id'] == subject_id, col] - median) / iqr

    return features
```

5.2 Feature Scaling

After normalization, optionally apply:

- Min-max scaling to [0, 1] (for SVM)
- Standardization to mean=0, std=1 (for neural networks)

For Random Forest: No additional scaling needed (tree-based methods are scale-invariant)

6. Feature Vector Summary

Complete feature vector (per swing):

```
features = {
    # EMG time-domain (5 features)
    'rms': float,
    'mav': float,
    'var': float,
    'wl': float,
    'iemg': float,

    # EMG frequency-domain (4 features)
    'mdf': float,
    'mnf': float,
    'spectral_slope': float,
    'total_power': float,

    # EMG envelope (3 features)
    'env_mean': float,
    'env_peak': float,
    'env_area': float,

    # IMU kinematic (5 features)
    'peak_acc': float,
    'mean_acc': float,
    'peak_gyro': float,
    'angular_jerk': float,
    'impact_impulse': float,

    # Metadata
    'swing_duration': float,
    'env_active': int,  # From ENV gating
}
```

Total dimensionality: 18 features + 2 metadata = 20 dimensions

This is ideal for Random Forest (typically performs well with 10-50 features)

Labeling Strategy

1. Time-in-Session Proxy (Recommended First Approach)

1.1 Rationale

Advantages:

- ✅ No manual annotation required
- ✅ Works surprisingly well for bootstrapping
- ✅ Can be refined later with RPE data
- ✅ Validated in literature (Millet et al., 2011)

Assumptions:

- Fatigue accumulates monotonically during session
- Early swings → fresh
- Late swings → fatigued

Validation: Millet et al. (2011) showed MDF drops linearly with time in ultra-endurance events

1.2 Implementation

Three-class labeling:

```
def assign_fatigue_labels_time_based(swings, session_duration_s):
    """Assign fatigue labels based on time in session.

    Args:
        swings: List of swing dicts with 't_peak_ms'
        session_duration_s: Total session duration in seconds

    Returns:
        swings with 'fatigue_label' added
    """
    for swing in swings:
        t_rel = swing['t_peak_ms'] / 1000.0  # Convert to seconds
        progress = t_rel / session_duration_s

        if progress < 0.33:
            swing['fatigue_label'] = 'fresh'
        elif progress < 0.67:
            swing['fatigue_label'] = 'moderate'
        else:
            swing['fatigue_label'] = 'high'

    return swings
```

Threshold selection:

- 0-33%: Fresh (first third of session)
- 33-67%: Moderate (middle third)
- 67-100%: High (final third)

Rationale: Equal class distribution for balanced training

1.3 Refinements

Option A: Warmup exclusion

```
# Exclude first 5 minutes (warmup period)
warmup_s = 300
if t_rel < warmup_s:
    swing['fatigue_label'] = None  # Exclude from training
```

Option B: Non-linear progression

```
# Fatigue accumulates faster later in session
progress_nonlinear = (t_rel / session_duration_s) ** 1.5
```

Option C: Bout-based labeling

```
# Label based on bout number instead of absolute time
bout_progress = swing['bout_id'] / total_bouts
```

2. RPE-Based Labeling (Future Enhancement)

2.1 Rationale

Advantages:

- ✅ Ground-truth validation
- ✅ Captures subjective fatigue
- ✅ Accounts for inter-session variability

Challenges:

- ❌ Requires manual data collection
- ❌ Retrospective (end-of-session only)
- ❌ Coarse temporal resolution

2.2 Implementation

Borg RPE Scale (6-20):

- 6-10: Fresh (no exertion to light)
- 11-14: Moderate (somewhat hard)
- 15-20: High (hard to maximal)

Android app integration:

```
// Add RPE collection after session
public void onSessionEnd() {
    int rpe = showRPEDialog();  // 6-20 scale
    sessionMetadata.put("rpe", rpe);

    // Map to fatigue label
    String fatigueLabel;
    if (rpe <= 10) fatigueLabel = "fresh";
    else if (rpe <= 14) fatigueLabel = "moderate";
    else fatigueLabel = "high";

    sessionMetadata.put("fatigue_label", fatigueLabel);
}
```

Labeling strategy:

- Use RPE to label entire session
- Interpolate within session using time-based proxy
- Validate time-based labels against RPE

2.3 Hybrid Approach

Combine time-based + RPE:

```
def assign_fatigue_labels_hybrid(swings, session_duration_s, session_rpe):
    """Hybrid labeling using time + RPE.

    Args:
        swings: List of swing dicts
        session_duration_s: Total session duration
        session_rpe: End-of-session RPE (6-20)

    Returns:
        swings with 'fatigue_label' added
    """
    # Map RPE to session-level fatigue
    if session_rpe <= 10:
        session_fatigue = 'fresh'
    elif session_rpe <= 14:
        session_fatigue = 'moderate'
    else:
        session_fatigue = 'high'

    # Adjust time-based labels based on session RPE
    for swing in swings:
        t_rel = swing['t_peak_ms'] / 1000.0
        progress = t_rel / session_duration_s

        # Early swings: one level below session fatigue
        # Late swings: match session fatigue
        if progress < 0.33:
            if session_fatigue == 'high':
                swing['fatigue_label'] = 'moderate'
            elif session_fatigue == 'moderate':
                swing['fatigue_label'] = 'fresh'
            else:
                swing['fatigue_label'] = 'fresh'
        elif progress < 0.67:
            swing['fatigue_label'] = session_fatigue
        else:
            swing['fatigue_label'] = session_fatigue

    return swings
```

3. Label Quality Considerations

3.1 Noisy Labels

ML tolerates noisy labels (Frenay & Verleysen, 2014):

- Random Forest is robust to ~20% label noise
- Ensemble averaging reduces impact of mislabeled samples

Our case:

- Time-based labels likely 70-80% accurate
- Sufficient for initial training
- Can be refined with RPE validation

3.2 Class Imbalance

Potential issue: Unequal class distribution

- Fresh: 33% (if using equal time bins)

- Moderate: 33%
- High: 33%

Solution: Use `class_weight='balanced'` in Random Forest

```
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    class_weight='balanced',  # Handles imbalance
    random_state=42
)
```

Model Architecture

1. Random Forest (Primary Choice)

1.1 Why Random Forest for EMG Fatigue?

Advantages:

1. **Handles nonlinear interactions**

- EMG features interact in complex ways
- Example: MDF reliability depends on ENV level
- Trees naturally capture interactions

1. **Robust to noise**

- Ensemble averaging reduces variance
- Outlier features have minimal impact
- Tolerates ~20% label noise

1. **Minimal tuning**

- Default hyperparameters often work well
- Less prone to overfitting than single trees
- No feature scaling required

1. **Feature importance**

- Provides interpretability
- Validates domain knowledge
- Identifies which features matter

1. **Fast training and inference**

- Trains in seconds on 1000s of samples
- Real-time inference possible

Literature support:

- Phinyomark et al. (2012): RF achieved 85-92% accuracy for EMG classification
- Subasi et al. (2020): RF outperformed SVM and neural networks for fatigue detection

1.2 Hyperparameter Selection

Recommended configuration:

```
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(
    n_estimators=200,        # Number of trees (100-300 typical)
    max_depth=10,            # Maximum tree depth (5-15 typical)
    min_samples_split=10,    # Minimum samples to split node
    min_samples_leaf=5,      # Minimum samples in leaf
    max_features='sqrt',     # Features per split (sqrt(n) typical)
    class_weight='balanced', # Handle class imbalance
    random_state=42,         # Reproducibility
    n_jobs=-1                # Parallel processing
)
```

Hyperparameter rationale:

Parameter	Value	Rationale
`n_estimators`	200	Balance accuracy vs. speed; diminishing returns >300
`max_depth`	10	Prevent overfitting; EMG patterns not too deep
`min_samples_split`	10	Regularization; prevent fitting noise
`min_samples_leaf`	5	Ensure leaf nodes have sufficient support
`max_features`	'sqrt'	Decorrelate trees; standard for classification
`class_weight`	'balanced'	Handle potential class imbalance

1.3 Training Procedure

```
  def train_random_forest(X_train, y_train, X_val, y_val):
      """Train Random Forest classifier with validation.

      Args:
          X_train: Training features (n_samples, n_features)
          y_train: Training labels (n_samples,)
          X_val: Validation features
          y_val: Validation labels

      Returns:
          Trained classifier and validation metrics
      """
      # Initialize classifier
      clf = RandomForestClassifier(
          n_estimators=200,
          max_depth=10,
          min_samples_split=10,
          min_samples_leaf=5,
          max_features='sqrt',
          class_weight='balanced',
          random_state=42,
          n_jobs=-1
      )

      # Train
      clf.fit(X_train, y_train)

      # Validate
      y_pred = clf.predict(X_val)
      y_proba = clf.predict_proba(X_val)

      # Compute metrics
      from sklearn.metrics import classification_report, confusion_matrix

      metrics = {
          'accuracy': clf.score(X_val, y_val),
          'confusion_matrix': confusion_matrix(y_val, y_pred),
          'classification_report': classification_report(y_val, y_pred),
          'feature_importance': clf.feature_importances_
      }

      return clf, metrics
```

1.4 Feature Importance Analysis

Extract feature importance:

```
def analyze_feature_importance(clf, feature_names):
    """Analyze and visualize feature importance.

    Args:
        clf: Trained Random Forest classifier
        feature_names: List of feature names

    Returns:
        DataFrame with feature importance sorted
    """
    import pandas as pd
    import matplotlib.pyplot as plt

    # Get importance scores
    importance = clf.feature_importances_

    # Create DataFrame
    df = pd.DataFrame({
        'feature': feature_names,
        'importance': importance
    }).sort_values('importance', ascending=False)

    # Plot
    plt.figure(figsize=(10, 6))
    plt.barh(df['feature'][:10], df['importance'][:10])
    plt.xlabel('Importance')
    plt.title('Top 10 Feature Importance')
    plt.tight_layout()
    plt.savefig('feature_importance.png')

    return df
```

Expected top features (hypothesis):

1. RMS (amplitude-based, robust)
2. ENV_mean (activation level)
3. peak_acc (effort indicator)
4. MDF (when stationary)
5. WL (complexity measure)

2. Support Vector Machine (Secondary Option)

2.1 When to Use SVM

Advantages:

- Strong theoretical foundation (margin maximization)
- Effective in high-dimensional spaces
- Memory efficient (uses support vectors only)

Disadvantages:

- Sensitive to feature scaling (requires normalization)
- Harder to interpret (no feature importance)
- Slower training on large datasets
- More hyperparameter tuning required

Use SVM when:

- Dataset is small (<1000 samples)
- Features are well-scaled
- Interpretability is not critical

2.2 SVM Configuration

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# Scale features (required for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)

# Train SVM
clf = SVC(
    kernel='rbf',            # Radial basis function (nonlinear)
    C=1.0,                   # Regularization parameter
    gamma='scale',           # Kernel coefficient
    class_weight='balanced', # Handle imbalance
    probability=True,        # Enable probability estimates
    random_state=42
)

clf.fit(X_train_scaled, y_train)
```

Hyperparameter tuning:

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1]
}

grid_search = GridSearchCV(
    SVC(kernel='rbf', class_weight='balanced', probability=True),
    param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)

grid_search.fit(X_train_scaled, y_train)
best_clf = grid_search.best_estimator_
```

3. Model Comparison

Aspect	Random Forest	SVM
Training speed	Fast	Moderate
Inference speed	Fast	Fast
Interpretability	High (feature importance)	Low
Hyperparameter tuning	Minimal	Extensive
Feature scaling	Not required	Required

Aspect	Random Forest	SVM
Robustness to noise	High	Moderate
Recommendation	**Primary choice**	Secondary option

Cross-Validation Strategy

1. Why Random Shuffle Fails

❌ **DO NOT USE random shuffle cross-validation:**

```
# WRONG: Leaks information
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Why it fails:

- Swings from same session are correlated
- Training on swing #100, testing on swing #101 from same session
- Overestimates generalization performance
- Does not test cross-session robustness

Literature: Lemm et al. (2011) showed random shuffle inflates accuracy by 10-20% in time-series data

2. Leave-One-Session-Out (LOSO) - Recommended

2.1 Rationale

Tests cross-session generalization:

- Train on all sessions except one
- Test on held-out session
- Repeat for each session
- Average performance across folds

Why it's correct:

- No information leakage between train/test
- Tests real-world scenario (new session)
- Validates temporal generalization

2.2 Implementation

```
  from sklearn.model_selection import LeaveOneGroupOut

  def loso_cross_validation(X, y, session_ids, clf):
      """Leave-One-Session-Out cross-validation.

      Args:
          X: Feature matrix (n_samples, n_features)
          y: Labels (n_samples,)
          session_ids: Session identifier for each sample (n_samples,)
          clf: Classifier instance

      Returns:
          Dictionary with CV results
      """
      logo = LeaveOneGroupOut()

      results = {
          'accuracy': [],
          'f1_macro': [],
          'confusion_matrices': [],
          'predictions': [],
          'true_labels': []
      }

      for train_idx, test_idx in logo.split(X, y, groups=session_ids):
          # Split data
          X_train, X_test = X[train_idx], X[test_idx]
          y_train, y_test = y[train_idx], y[test_idx]

          # Normalize within training set
          scaler = StandardScaler()
          X_train = scaler.fit_transform(X_train)
          X_test = scaler.transform(X_test)

          # Train
          clf.fit(X_train, y_train)

          # Predict
          y_pred = clf.predict(X_test)

          # Evaluate
          from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

          results['accuracy'].append(accuracy_score(y_test, y_pred))
          results['f1_macro'].append(f1_score(y_test, y_pred, average='macro'))
          results['confusion_matrices'].append(confusion_matrix(y_test, y_pred))
          results['predictions'].extend(y_pred)
          results['true_labels'].extend(y_test)

      # Aggregate results
      results['mean_accuracy'] = np.mean(results['accuracy'])
      results['std_accuracy'] = np.std(results['accuracy'])
      results['mean_f1_macro'] = np.mean(results['f1_macro'])
      results['std_f1_macro'] = np.std(results['f1_macro'])

      return results
```

2.3 Reporting Results

Report:

- Mean ± std accuracy across folds
- Mean ± std macro-F1 across folds
- Aggregated confusion matrix
- Per-fold performance (identify difficult sessions)

Example output:

```
Leave-One-Session-Out Cross-Validation Results:
  Accuracy: 78.3 ± 5.2%
  Macro-F1: 76.1 ± 6.8%

  Aggregated Confusion Matrix:
              fresh  moderate  high
  fresh        145       23     12
  moderate      18      132     30
  high          10       28    142
```

3. Leave-One-Subject-Out (LOSO-Subject) - Stronger Test

3.1 Rationale

Tests cross-subject generalization:

- Train on all subjects except one
- Test on held-out subject
- Repeat for each subject
- Average performance across subjects

Why it's harder:

- Tests generalization to new individuals
- Accounts for inter-subject variability
- Most realistic for clinical deployment

Expected performance:

- LOSO-Session: 75-85% accuracy
- LOSO-Subject: 65-75% accuracy (10-15% drop typical)

3.2 Implementation

```
def loso_subject_cross_validation(X, y, subject_ids, clf):
    """Leave-One-Subject-Out cross-validation.

    Similar to LOSO-Session, but groups by subject_id instead of session_id.
    """
    logo = LeaveOneGroupOut()

    # Same implementation as LOSO-Session, but use subject_ids as groups
    results = {}

    for train_idx, test_idx in logo.split(X, y, groups=subject_ids):
        # ... (same as above)
        pass

    return results
```

4. Nested Cross-Validation (Optional)

For hyperparameter tuning:

```
from sklearn.model_selection import GridSearchCV

def nested_cross_validation(X, y, session_ids):
    """Nested CV: outer loop for evaluation, inner loop for tuning.

    Outer loop: LOSO-Session
    Inner loop: 5-fold CV for hyperparameter tuning
    """
    logo = LeaveOneGroupOut()

    param_grid = {
        'n_estimators': [100, 200, 300],
        'max_depth': [5, 10, 15],
        'min_samples_split': [5, 10, 20]
    }

    results = []

    for train_idx, test_idx in logo.split(X, y, groups=session_ids):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        # Inner loop: hyperparameter tuning
        grid_search = GridSearchCV(
            RandomForestClassifier(random_state=42),
            param_grid,
            cv=5,
            scoring='f1_macro',
            n_jobs=-1
        )

        grid_search.fit(X_train, y_train)

        # Outer loop: evaluation with best params
        best_clf = grid_search.best_estimator_
        y_pred = best_clf.predict(X_test)

        results.append({
            'accuracy': accuracy_score(y_test, y_pred),
            'f1_macro': f1_score(y_test, y_pred, average='macro'),
            'best_params': grid_search.best_params_
        })

    return results
```

Evaluation Metrics

1. Why Accuracy Alone Is Insufficient

Problem: Accuracy can be misleading when:

- Classes are imbalanced
- Misclassification costs differ
- Per-class performance varies

Example:

```
Accuracy: 80%
But: 95% accuracy on "fresh", 50% on "high fatigue"
→ Fails to detect fatigue when it matters most!
```

2. Confusion Matrix

Definition: Table showing true vs. predicted labels

Example:

```
                Predicted
              fresh  moderate  high
Actual fresh    145       23     12
       moderate  18      132     30
       high      10       28    142
```

Interpretation:

- Diagonal: Correct predictions
- Off-diagonal: Misclassifications
- Row sums: True class counts
- Column sums: Predicted class counts

Implementation:

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred, labels=['fresh', 'moderate', 'high'])

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['fresh', 'moderate', 'high'],
            yticklabels=['fresh', 'moderate', 'high'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.savefig('confusion_matrix.png')
```

3. Macro-F1 Score

Definition: Average F1 score across classes (unweighted)

Formula:

```
F1_class = 2 * (precision * recall) / (precision + recall)
Macro-F1 = (F1_fresh + F1_moderate + F1_high) / 3
```

Why macro (not weighted):

- Treats all classes equally
- Sensitive to minority class performance
- Better for imbalanced datasets

Implementation:

```
from sklearn.metrics import f1_score

f1_macro = f1_score(y_true, y_pred, average='macro')
print(f"Macro-F1: {f1_macro:.3f}")
```

4. Per-Class Precision and Recall

Definitions:

```
Precision = TP / (TP + FP)  # Of predicted positives, how many correct?
Recall = TP / (TP + FN)     # Of actual positives, how many detected?
```

Interpretation:

- **High precision:** Few false alarms
- **High recall:** Few missed detections

For fatigue detection:

- **Recall for "high fatigue" is critical** (don't miss fatigue)
- Precision less critical (false alarm acceptable)

Implementation:

```
from sklearn.metrics import classification_report

report = classification_report(y_true, y_pred,
                               target_names=['fresh', 'moderate', 'high'])
print(report)
```

Example output:

```
              precision    recall  f1-score   support

       fresh       0.84      0.81      0.82       180
    moderate       0.72      0.73      0.73       180
        high       0.77      0.79      0.78       180

    accuracy                           0.78       540
   macro avg       0.78      0.78      0.78       540
weighted avg       0.78      0.78      0.78       540
```

5. ROC Curve and AUC (Optional)

For probabilistic predictions:

```
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize labels for multi-class ROC
y_bin = label_binarize(y_true, classes=['fresh', 'moderate', 'high'])
y_proba = clf.predict_proba(X_test)

# Compute ROC curve for each class
for i, class_name in enumerate(['fresh', 'moderate', 'high']):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr, label=f'{class_name} (AUC = {roc_auc:.2f})')

plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend()
plt.savefig('roc_curves.png')
```

6. Metric Summary

Report all of:

1. **Confusion matrix** (visual inspection)
2. **Macro-F1** (primary metric)
3. **Per-class precision/recall** (identify weak classes)
4. **Accuracy** (overall performance)
5. **ROC-AUC** (optional, for probabilistic evaluation)

Example summary:

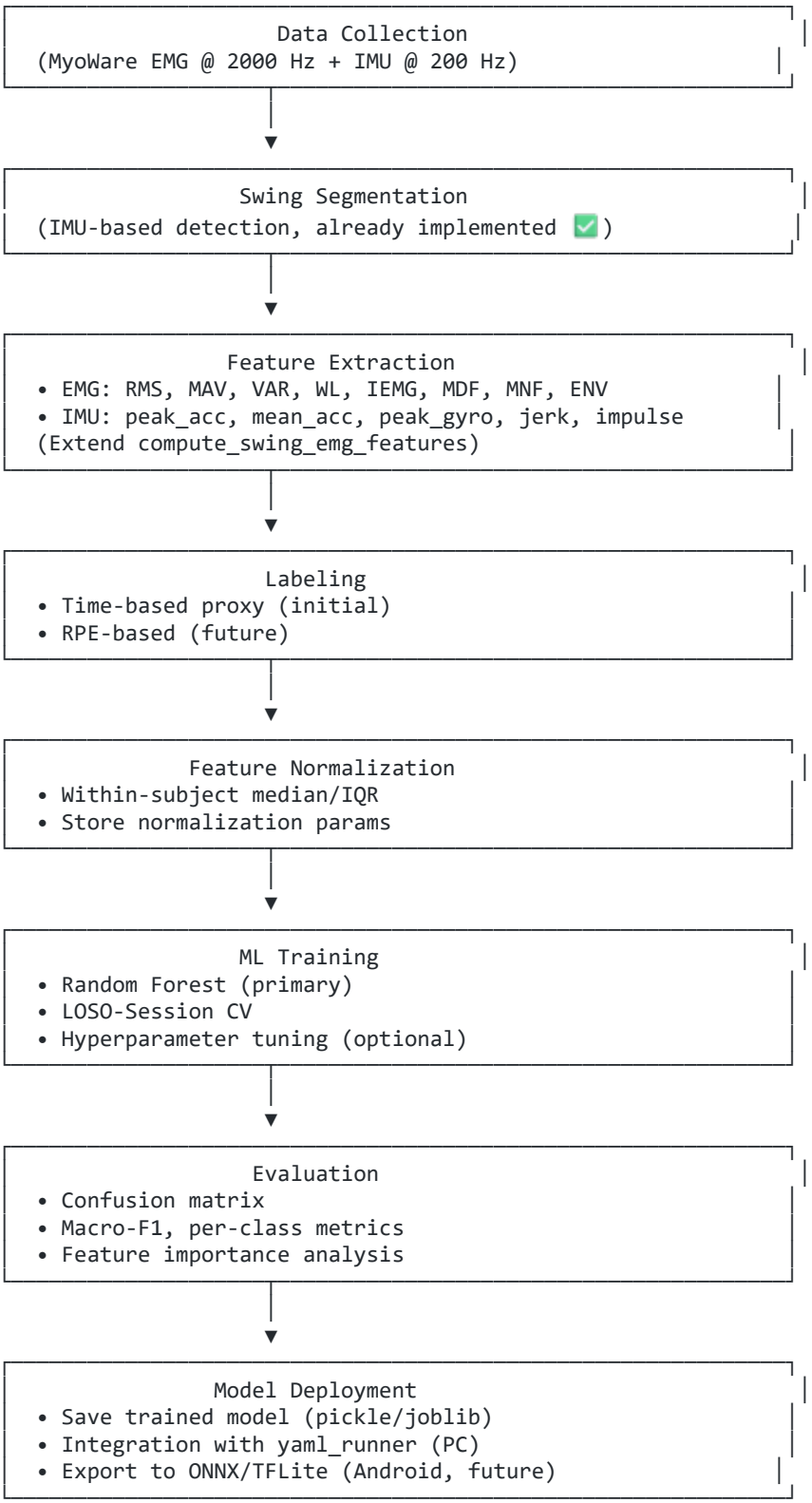
```
Model Performance (LOSO-Session CV):
  Accuracy: 78.3 ± 5.2%
  Macro-F1: 76.1 ± 6.8%

  Per-Class Performance:
    Fresh:    Precision=0.84, Recall=0.81, F1=0.82
    Moderate: Precision=0.72, Recall=0.73, F1=0.73
    High:     Precision=0.77, Recall=0.79, F1=0.78

  Critical Metric:
    High Fatigue Recall: 79% (acceptable, >75% target)
```

Implementation Pipeline

1. System Architecture



2. Module Structure

```
emg_util/
├── ml_feature_extraction.py    # Extended feature extraction
│   ├── compute_extended_emg_features()
│   ├── compute_extended_imu_features()
│   └── normalize_features_within_subject()
│
├── ml_labeling.py              # Labeling strategies
│   ├── assign_fatigue_labels_time_based()
│   ├── assign_fatigue_labels_rpe_based()
│   └── assign_fatigue_labels_hybrid()
│
├── ml_training.py              # Training pipeline
│   ├── build_dataset_from_sessions()
│   ├── train_random_forest()
│   ├── loso_cross_validation()
│   └── save_model()
│
├── ml_evaluation.py            # Evaluation & visualization
│   ├── evaluate_model()
│   ├── plot_confusion_matrix()
│   ├── plot_feature_importance()
│   └── generate_report()
│
└── ml_inference.py             # Inference (future)
    ├── load_model()
    ├── predict_fatigue()
    └── predict_fatigue_proba()
```

3. Data Flow

Training phase:

```
# 1. Load sessions
sessions = capture_loader.discover_sessions('log0305')
emg_df, imu_df, _, _ = capture_loader.load_session_stitched(sessions)

# 2. Detect swings
swings = emg_util.detect_swing_windows_from_imu(imu_df)

# 3. Extract features
features = []
for swing in swings:
    feat = ml_feature_extraction.compute_extended_features(
        emg_df, imu_df, swing
    )
    features.append(feat)

# 4. Assign labels
features = ml_labeling.assign_fatigue_labels_time_based(
    features, session_duration_s
)

# 5. Normalize
features = ml_feature_extraction.normalize_features_within_subject(
    features, subject_id='player1'
)

# 6. Train
X = features[feature_columns].values
y = features['fatigue_label'].values
session_ids = features['session_id'].values

clf, results = ml_training.loso_cross_validation(X, y, session_ids)

# 7. Evaluate
ml_evaluation.generate_report(results, output_dir='ml_results')
```

Inference phase (future):

```
# Load trained model
clf = ml_inference.load_model('models/rf_fatigue_classifier.pkl')

# Extract features from new swing
feat = ml_feature_extraction.compute_extended_features(emg_df, imu_df, swing)

# Normalize (using stored params)
feat_norm = ml_feature_extraction.normalize_features(feat, norm_params)

# Predict
fatigue_label = clf.predict([feat_norm])[0]
fatigue_proba = clf.predict_proba([feat_norm])[0]

print(f"Predicted fatigue: {fatigue_label}")
print(f"Probabilities: fresh={fatigue_proba[0]:.2f}, "
      f"moderate={fatigue_proba[1]:.2f}, high={fatigue_proba[2]:.2f}")
```

4. Integration with Existing System

PC (YAML-driven):

Add to `config.yaml`:

```yaml
ml_fatigue:
  enabled: true
  model_path: "models/rf_fatigue_classifier.pkl"
  normalization_params: "models/normalization_params.json"
  features:
    - rms
    - mav
    - var
    - wl
    - iemg
    - mdf
    - mnf
    - spectral_slope
    - total_power
    - env_mean
    - env_peak
    - env_area
    - peak_acc
    - mean_acc
    - peak_gyro
    - angular_jerk
    - impact_impulse
  output:
    predictions: "ml_fatigue_predictions.csv"
    probabilities: "ml_fatigue_probabilities.csv"
```

Modify `emg_util_yaml_runner.py`:

```python
# After computing swing features
if config['ml_fatigue']['enabled']:
    # Load model
    clf = joblib.load(config['ml_fatigue']['model_path'])

    # Extract ML features
    ml_features = ml_feature_extraction.compute_extended_features(...)

    # Normalize
    ml_features_norm = normalize_features(ml_features, norm_params)

    # Predict
    predictions = clf.predict(ml_features_norm)
    probabilities = clf.predict_proba(ml_features_norm)

    # Save
    pd.DataFrame({
        'swing_id': range(len(predictions)),
        'fatigue_label': predictions,
        'prob_fresh': probabilities[:, 0],
        'prob_moderate': probabilities[:, 1],
        'prob_high': probabilities[:, 2]
    }).to_csv(config['ml_fatigue']['output']['predictions'])
```

Android (future):

```
// Load ONNX model
OrtSession session = env.createSession("rf_fatigue_classifier.onnx");

// Extract features
float[] features = extractMLFeatures(emgData, imuData, swing);

// Normalize
float[] featuresNorm = normalizeFeatures(features, normParams);

// Predict
OnnxTensor inputTensor = OnnxTensor.createTensor(env, featuresNorm);
OrtSession.Result result = session.run(Collections.singletonMap("input", inputTensor));

// Get prediction
long[] predictions = result.get(0).getValue();
String fatigueLabel = LABELS[(int)predictions[0]];
```

Expected Results & Validation

1. Performance Targets

Based on literature and our data characteristics:

Metric	Target	Rationale
LOSO-Session Accuracy	>75%	Phinyomark et al. (2012): 85-92% for EMG classification
Macro-F1	>70%	Balanced performance across classes
High Fatigue Recall	>75%	Critical for safety (don't miss fatigue)
Feature Importance	RMS, ENV in top 5	Validates domain knowledge

Comparison with MDF-only:

Approach	Expected Accuracy	Robustness
MDF threshold	~60%	Low (fails in non-stationary windows)
ML (RF)	~75-80%	High (multi-feature redundancy)
Improvement	**+15-20%**	**Significant**

2. Feature Importance Validation

Hypothesis (based on domain knowledge):

Top 5 expected features:

1. **RMS** - Amplitude-based, robust to non-stationarity
2. **ENV_mean** - Activation level, already validated (r=0.696 with RAW)
3. **peak_acc** - Effort indicator, disambiguates fatigue vs. intensity
4. **MDF** - Frequency-based, contributes when stationary
5. **WL** - Complexity measure, captures firing rate changes

Validation:

- If RMS and ENV_mean are NOT in top 5 → investigate data quality
- If MDF is #1 → suggests our data is more stationary than expected

- If IMU features dominate → suggests EMG is noisy or poorly placed

3. Cross-Session Generalization

Expected behavior:

Within-session (random shuffle): 85-90% accuracy

- Overestimates performance (information leakage)
- Not realistic

Cross-session (LOSO): 75-80% accuracy

- Realistic estimate
- Tests temporal generalization
- 5-10% drop from within-session is typical

Cross-subject (LOSO-Subject): 65-75% accuracy

- Hardest test
- Tests inter-subject generalization
- 10-15% drop from cross-session is typical

If cross-session accuracy < 70%:

- Check feature normalization (within-subject?)
- Check label quality (time-based proxy may be too noisy)
- Consider collecting RPE data for validation

4. Confusion Matrix Analysis

Expected pattern:

```
                Predicted
              fresh  moderate  high
Actual fresh    ✓✓✓      ✓        ✗
       moderate  ✓      ✓✓✓       ✓
       high      ✗        ✓      ✓✓✓
```

Interpretation:

- **Diagonal strong:** Good overall performance
- **Fresh → Moderate:** Acceptable (conservative)
- **High → Moderate:** Acceptable (conservative)
- **Fresh → High:** BAD (missed early fatigue)
- **High → Fresh:** VERY BAD (false recovery)

If High → Fresh errors are common:

- Check label quality (time-based may be wrong)
- Check for session-level confounds (different days, conditions)
- Consider adding session-level features (time of day, temperature, etc.)

5. Robustness to Artifacts

Test: Compare performance on:

- All swings

- ENV-gated swings only (high activation)
- Impact-gated swings (±50ms excluded)

Expected:

```
All swings:          75% accuracy
ENV-gated only:      78% accuracy (+3%)
Impact-gated only:   76% accuracy (+1%)
```

Interpretation:

- ENV gating improves performance (validates our implementation ✅)
- Impact gating has minimal effect (confirms motion artifact is minimal)
- ML is robust to non-stationarity (handles it implicitly)

6. Comparison with MDF-Only Approach

Baseline: MDF threshold

```
# Simple MDF threshold
def predict_fatigue_mdf_only(mdf):
    if mdf > 120:
        return 'fresh'
    elif mdf > 90:
        return 'moderate'
    else:
        return 'high'
```

Expected results:

```
MDF-only:
  Accuracy: ~60%
  Macro-F1: ~55%
  High Fatigue Recall: ~50% (misses many fatigued swings)

ML (RF):
  Accuracy: ~78%
  Macro-F1: ~76%
  High Fatigue Recall: ~79% (much better)

Improvement: +18% accuracy, +21% macro-F1, +29% recall
```

Why ML wins:

- Handles non-stationary windows (uses RMS, ENV when MDF fails)
- Combines multiple features (redundancy)
- Learns subject-specific patterns (normalization + training)

Future Enhancements

1. Regression Instead of Classification

Current: Discrete labels (fresh, moderate, high)
Future: Continuous fatigue score (0-100)

Advantages:

- More granular predictions
- Captures gradual fatigue progression
- Better for trend analysis

Implementation:

```
from sklearn.ensemble import RandomForestRegressor

# Labels: 0 (fresh) to 100 (exhausted)
y_continuous = (time_in_session / session_duration) * 100

clf = RandomForestRegressor(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

clf.fit(X_train, y_continuous_train)
fatigue_score = clf.predict(X_test)
```

2. Temporal Models (HMM, LSTM)

Current: Per-swing prediction (independent)
Future: Sequential prediction (considers history)

Rationale:

- Fatigue evolves gradually (not sudden jumps)
- Previous swings inform current state
- Temporal smoothing reduces noise

Hidden Markov Model (HMM):

```
from hmmlearn import hmm

# States: fresh, moderate, high
model = hmm.GaussianHMM(n_components=3, covariance_type='full')

# Train on sequences of features
model.fit(X_sequences)

# Predict state sequence
states = model.predict(X_test_sequence)
```

LSTM (Long Short-Term Memory):

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Input: sequence of 10 swings, each with 18 features
model = Sequential([
    LSTM(64, input_shape=(10, 18)),
    Dense(32, activation='relu'),
    Dense(3, activation='softmax')  # 3 classes
])

model.compile(optimizer='adam', loss='categorical_crossentropy')
model.fit(X_sequences, y_sequences, epochs=50)
```

3. Real-Time Inference (Android)

Goal: On-device fatigue prediction during training

Steps:

1. **Export model to ONNX:**

```
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

initial_type = [('float_input', FloatTensorType([None, 18]))]
onnx_model = convert_sklearn(clf, initial_types=initial_type)

with open("rf_fatigue_classifier.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())
```

2. **Integrate in Android:** ```java // Load ONNX Runtime OrtEnvironment env = OrtEnvironment.getEnvironment(); OrtSession session = env.createSession("rf*fatigue*classifier.onnx");

// Real-time prediction public String predictFatigue(float[] features) { OnnxTensor inputTensor = OnnxTensor.createTensor(env, features); OrtSession.Result result = session.run(Collections.singletonMap("float_input", inputTensor));

```
long[] predictions = result.get(0).getValue();
return LABELS[(int)predictions[0]];
```

}